



UNIVERSIDAD POLITÉCNICA DE CHIAPAS

Maestría en Ciencias e Innovación Tecnológica

Asignatura

Lógica y Programación

Protocolo de Proyecto

Optimización Concurrente de un Sistema de
Sincronización de Semáforos

Integrantes

Iván Antonio Rincón Grajales
Viviana López Rojo
Ramsés Alejandro Camas Nájera
Guillermo Tinoco Ramos
Montserrat Viridiana Ramos López
Jesús Rodrigo Mijangos Gordillo
Carmen Anel Vilca Mamani

Tuxtla Gutiérrez, Chiapas

2026

Resumen

La sincronización eficiente de semáforos constituye uno de los problemas fundamentales en el diseño de sistemas de movilidad para ciudades inteligentes, debido a su impacto directo en la congestión vehicular, los tiempos de traslado y el consumo energético en entornos urbanos.

Este trabajo presenta el análisis y rediseño arquitectónico de un sistema computacional destinado a procesar información de tráfico proveniente de múltiples intersecciones. El sistema se modela mediante un pipeline de procesamiento compuesto por cuatro etapas principales: adquisición de datos de sensores, análisis de métricas de tráfico, optimización del plan semafórico y actualización del estado del controlador.

Inicialmente se analiza una implementación base denominada **Estado P**, la cual ejecuta el procesamiento de cada intersección de forma estrictamente secuencial. Este enfoque introduce limitaciones importantes de escalabilidad cuando el número de intersecciones aumenta, debido a que el tiempo total de ejecución crece linealmente con el tamaño del sistema.

A partir de este análisis se propone una reestructuración arquitectónica denominada **Estado Q**, basada en programación orientada a objetos, patrones de diseño y técnicas de concurrencia. La arquitectura propuesta emplea un modelo híbrido que combina *AsyncIO* para operaciones de entrada/salida (I/O-bound) y multiprocesamiento para tareas intensivas de cómputo (CPU-bound).

La evaluación experimental demuestra que la arquitectura concurrente reduce significativamente la latencia de procesamiento y aumenta el throughput del sistema al permitir el procesamiento simultáneo de múltiples intersecciones.

Repositorio del proyecto:

https://github.com/RamsesCamas/MCIT_LOG_PROG_Proyecto_Final.git

1. Introducción

El crecimiento del tráfico vehicular en entornos urbanos representa uno de los principales desafíos para la movilidad en ciudades modernas. En el contexto de las ciudades inteligentes, los sistemas de control de tráfico buscan optimizar el flujo vehicular mediante el uso de sensores, sistemas de monitoreo y algoritmos de optimización capaces de ajustar dinámicamente los tiempos de los semáforos.

Uno de los problemas centrales en este tipo de sistemas consiste en la coordinación eficiente de múltiples intersecciones de tráfico. Cada intersección debe analizar continuamente información proveniente de sensores o cámaras, estimar el estado del tráfico y calcular un plan semafórico adecuado que minimice la congestión y mejore la fluidez vehicular. Este tipo de problemas suele modelarse como sistemas de procesamiento de datos donde múltiples unidades deben analizar y transformar información de manera continua.

Desde el punto de vista computacional, este tipo de sistemas plantea desafíos importantes relacionados con el rendimiento y la escalabilidad. A medida que el número de intersecciones aumenta, el sistema debe ser capaz de procesar una mayor cantidad de datos en intervalos de tiempo cada vez más reducidos. En sistemas implementados de manera secuencial, el tiempo total de procesamiento tiende a crecer de forma proporcional al tamaño del problema, lo que puede limitar la capacidad del sistema para operar en tiempo cercano al real.

En las últimas décadas, el desarrollo de arquitecturas multicore ha cambiado significativamente la forma en que se diseñan sistemas de alto rendimiento. En lugar de depender exclusivamente del incremento en la frecuencia de los procesadores, los sistemas modernos se basan en el uso de múltiples núcleos de procesamiento que permiten ejecutar tareas de forma concurrente o paralela [2]. Como consecuencia, el diseño de software ha experimentado una transición hacia modelos basados en concurrencia y paralelismo [3].

La concurrencia permite gestionar múltiples tareas de forma intercalada o simultánea, mientras que el paralelismo permite ejecutar varias operaciones al mismo tiempo utilizando diferentes unidades de procesamiento. Estos enfoques resultan especialmente útiles en sistemas donde existen múltiples tareas independientes o parcialmente independientes que pueden procesarse de manera simultánea [5].

El sistema analizado en este proyecto modela el procesamiento de cada intersección mediante un pipeline compuesto por cuatro etapas principales:

Read $\rightarrow$ Analyze $\rightarrow$ Optimize $\rightarrow$ Update

Estas etapas corresponden respectivamente a la adquisición de datos de tráfico, el análisis de métricas vehiculares, la generación de un plan semafórico y la actualización del estado del controlador del semáforo.

En la implementación inicial del sistema, denominada **Estado P**, todas las intersecciones son procesadas de manera estrictamente secuencial. Si la red urbana contiene n intersecciones, el tiempo total de ejecución del sistema puede aproximarse como:

$$T_P = \sum_{i=1}^n (t_r + t_a + t_o + t_u)$$

donde cada término representa el tiempo requerido por cada etapa del pipeline de procesamiento.

Este modelo implica que el tiempo total de ejecución crece linealmente con el número de intersecciones procesadas. En sistemas de gran escala, esta característica puede generar limitaciones importantes de rendimiento.

El análisis de sistemas paralelos ha demostrado que la mejora de rendimiento obtenida mediante paralelización depende de la fracción del programa que puede ejecutarse simultáneamente. Este principio fue formalizado por Amdahl en su conocida ley de paralelización, la cual establece un límite teórico para el speedup máximo alcanzable en un sistema paralelo [1].

Motivado por estas observaciones, el presente proyecto propone una reestructuración arquitectónica del sistema basada en programación orientada a objetos y en el uso de técnicas de concurrencia.

La nueva implementación, denominada **Estado Q**, introduce una arquitectura modular basada en el patrón *Pipeline* y en el patrón *Strategy*. Además, incorpora un modelo híbrido de concurrencia que combina programación asíncrona para operaciones I/O-bound y multi-procesamiento para tareas CPU-bound.

Este enfoque permite que múltiples intersecciones se procesen de manera concurrente, reduciendo el tiempo total de ejecución del sistema y mejorando su capacidad de escalamiento.

En las secciones siguientes se describe el planteamiento del problema, los objetivos del proyecto, el marco técnico que sustenta la propuesta, la arquitectura del sistema y la metodología utilizada para evaluar el impacto de la concurrencia en el rendimiento del procesamiento de intersecciones de tráfico.

2. Planteamiento del problema (PCQ)

2.1. Problema

Los sistemas de control de tráfico urbano deben procesar continuamente información proveniente de múltiples intersecciones para determinar los tiempos óptimos de los semáforos. Este procesamiento implica la adquisición de datos de sensores, el análisis de métricas vehiculares, la generación de planes de señalización y la actualización del estado de los controladores de tráfico.

En la implementación inicial del sistema analizado en este proyecto, denominada **Estado P**, el procesamiento de las intersecciones se realiza de manera estrictamente secuencial. Esto significa que cada intersección debe completar todas las etapas del pipeline antes de que la siguiente pueda comenzar su procesamiento.

Formalmente, si una red urbana contiene n intersecciones $I = \{i_1, i_2, \dots, i_n\}$, el procesamiento total del sistema puede representarse como:

$$P = F(i_1); F(i_2); \dots; F(i_n)$$

donde cada función $F(i)$ corresponde al pipeline de procesamiento de una intersección:

$$F(i) = read(i) \rightarrow analyze(i) \rightarrow optimize(i) \rightarrow update(i)$$

Bajo este modelo secuencial, el tiempo total de ejecución del sistema puede aproximarse como:

$$T_P = \sum_{i=1}^n (t_r + t_a + t_o + t_u)$$

donde:

- t_r corresponde al tiempo de lectura de sensores
- t_a corresponde al tiempo de análisis de métricas de tráfico
- t_o corresponde al tiempo de optimización del plan semafórico
- t_u corresponde al tiempo de actualización del controlador

Este modelo implica que el tiempo total de procesamiento crece linealmente con el número de intersecciones. En redes urbanas con un número elevado de intersecciones, esta característica puede generar latencias significativas y limitar la capacidad del sistema para responder en tiempo cercano al real.

Además, el análisis del comportamiento del sistema revela que una parte importante del tiempo de ejecución se encuentra asociada a operaciones de entrada y salida, como la lectura de sensores o la actualización de los controladores de semáforos. Estas operaciones presentan características típicas de tareas *I/O-bound*, lo que provoca que el sistema permanezca bloqueado esperando la finalización de dichas operaciones antes de continuar con el procesamiento.

Como consecuencia, el sistema presenta tres limitaciones principales:

- ausencia de paralelismo en el procesamiento de intersecciones
- subutilización de los recursos computacionales disponibles
- crecimiento lineal del tiempo de ejecución conforme aumenta el número de intersecciones

Estas limitaciones motivan la necesidad de analizar alternativas arquitectónicas que permitan mejorar el rendimiento del sistema.

2.2. Contexto

El sistema estudiado corresponde a un modelo computacional de control de tráfico urbano en el cual múltiples intersecciones deben procesar datos de forma continua para determinar el estado de sus semáforos.

Cada intersección ejecuta un pipeline compuesto por cuatro etapas principales:

1. adquisición de datos de sensores de tráfico
2. análisis de métricas vehiculares
3. cálculo del plan semafórico
4. actualización del estado del semáforo

Desde el punto de vista computacional, estas etapas presentan características distintas:

- las operaciones de lectura y actualización corresponden a tareas *I/O-bound*, debido a que implican comunicación con sensores o controladores
- las etapas de análisis y optimización corresponden a tareas *CPU-bound*, ya que requieren cálculos computacionales para procesar las métricas de tráfico

Esta combinación de tareas con características diferentes sugiere que el sistema podría beneficiarse de una arquitectura que aproveche técnicas de concurrencia y paralelismo.

El proyecto se desarrolla utilizando el lenguaje de programación Python y evalúa mecanismos de concurrencia disponibles en este entorno, incluyendo:

- **AsyncIO** para operaciones de entrada y salida
- **multiprocessing** para tareas intensivas de CPU

La propuesta consiste en rediseñar la arquitectura del sistema para permitir el procesamiento concurrente de múltiples intersecciones, manteniendo al mismo tiempo una estructura modular basada en programación orientada a objetos.

2.3. Formalización del pipeline mediante tripletas de Hoare

Para describir formalmente el comportamiento del sistema se emplea la notación de tripletas de Hoare, la cual permite expresar la relación entre una precondition, una operación y una postcondition. En términos generales, una tripleta de Hoare se representa como:

$$\{P\} C \{Q\}$$

donde P representa la condición inicial, C la instrucción o proceso ejecutado, y Q la condición final que debe cumplirse después de la ejecución.

En el contexto de este proyecto, cada intersección i se procesa mediante un pipeline compuesto por cuatro etapas: lectura de datos, análisis, optimización y actualización. Cada una de estas etapas puede formalizarse mediante una tripleta de Hoare.

Etapas de lectura La primera etapa del pipeline recibe la información de tráfico de una intersección y produce datos crudos provenientes de los sensores:

$$\{traffic_data(i)\} read(i) \{raw_data(i)\}$$

Esta tripleta expresa que, dado un conjunto de datos de tráfico asociados a la intersección i , la operación $read(i)$ produce una representación cruda de dichos datos.

Etapas de análisis La segunda etapa transforma los datos crudos en métricas relevantes para la toma de decisiones:

$$\{raw_data(i)\} analyze(i) \{features(i)\}$$

En esta etapa se extraen variables como densidad vehicular, flujo de tráfico y longitud de cola, las cuales constituyen las *features* utilizadas por el optimizador.

Etapas de optimización La tercera etapa toma las métricas calculadas y genera un plan semafórico:

$$\{features(i)\} \text{ optimize}(i) \{signal_plan(i)\}$$

Esta tripleta indica que, a partir de las características del tráfico de la intersección i , la operación $optimize(i)$ produce un plan de señalización que define parámetros como tiempo de luz verde, desfase temporal y número de fases.

Etapas de actualización La cuarta etapa aplica el plan generado al controlador del semáforo y produce el nuevo estado del sistema:

$$\{signal_plan(i)\} \text{ update}(i) \{light_state(i)\}$$

Aquí se expresa que, dado un plan semafórico válido, la operación $update(i)$ produce el estado final del semáforo para la intersección i .

Formalización del pipeline completo A partir de las tripletas anteriores, el pipeline completo para una intersección puede resumirse como:

$$\{traffic_data(i)\} F(i) \{light_state(i)\}$$

donde:

$$F(i) = update(i) \circ optimize(i) \circ analyze(i) \circ read(i)$$

Esta expresión representa la composición secuencial de las cuatro etapas del pipeline, desde la lectura de datos hasta la actualización final del semáforo.

Formalización del sistema global Si la red urbana contiene un conjunto de intersecciones $I = \{i_1, i_2, \dots, i_n\}$, entonces el sistema completo puede expresarse, a nivel global, como:

$$\{TrafficNetwork\} \text{ System } \{UpdatedTrafficLights\}$$

donde el sistema procesa todas las intersecciones de la red y genera el estado actualizado de los semáforos. Esta última formulación resume el comportamiento total del sistema, mientras

que las tripletas anteriores describen formalmente la lógica interna de cada etapa del pipeline.

2.4. Pregunta de investigación

A partir del análisis del sistema secuencial y de las limitaciones identificadas, surge la siguiente pregunta de investigación:

¿Cómo puede mejorarse el rendimiento de un sistema de procesamiento de intersecciones de tráfico mediante el uso de concurrencia y una arquitectura modular basada en pipelines?

En particular, el proyecto busca evaluar si la introducción de una arquitectura concurrente que combine operaciones asíncronas para tareas I/O-bound y multiprocesamiento para tareas CPU-bound puede reducir significativamente el tiempo total de procesamiento del sistema y mejorar su escalabilidad conforme aumenta el número de intersecciones.

El análisis de esta pregunta permitirá determinar en qué medida el uso de concurrencia puede optimizar el rendimiento del sistema sin afectar la correctitud del procesamiento del pipeline.

3. Objetivos

3.1. Objetivo general

Analizar y optimizar el rendimiento de un sistema de procesamiento de intersecciones de tráfico mediante el rediseño de su arquitectura computacional, incorporando una estructura modular basada en pipelines y mecanismos de concurrencia que permitan mejorar la escalabilidad y reducir el tiempo total de ejecución del sistema.

3.2. Objetivos específicos

- Analizar la arquitectura inicial del sistema (Estado P), identificando las etapas del pipeline de procesamiento y los posibles cuellos de botella asociados al modelo secuencial.
- Caracterizar el comportamiento computacional de las distintas etapas del sistema, distinguiendo entre tareas *I/O-bound* y tareas *CPU-bound* dentro del pipeline de procesamiento.
- Diseñar una arquitectura modular basada en programación orientada a objetos que permita desacoplar las responsabilidades del sistema mediante componentes independientes para adquisición, procesamiento, optimización y control.
- Implementar una arquitectura concurrente (Estado Q) que permita el procesamiento simultáneo de múltiples intersecciones mediante el uso de *AsyncIO* para operaciones de entrada y salida y *multiprocessing* para tareas intensivas de cómputo.
- Integrar patrones de diseño como *Pipeline* y *Strategy* con el fin de mejorar la extensibilidad del sistema y permitir la incorporación de diferentes algoritmos de optimización de señales de tráfico.
- Evaluar experimentalmente el rendimiento de la arquitectura propuesta mediante métricas como tiempo de ejecución, throughput y speedup, comparando los resultados obtenidos entre el modelo secuencial y el modelo concurrente.
- Analizar el impacto de la concurrencia en la escalabilidad del sistema conforme aumenta el número de intersecciones procesadas.

4. Marco técnico

El desarrollo de sistemas capaces de procesar grandes volúmenes de información en tiempos reducidos requiere el uso de arquitecturas computacionales eficientes y de técnicas adecuadas de concurrencia. En particular, los sistemas que procesan múltiples flujos de datos de forma simultánea deben diseñarse considerando la forma en que las tareas se distribuyen y ejecutan dentro del hardware disponible.

En las últimas décadas, el aumento en el rendimiento de los sistemas computacionales ha estado impulsado principalmente por la aparición de procesadores multicore, los cuales permiten ejecutar múltiples tareas de manera concurrente o paralela [2]. Este cambio en la arquitectura de hardware ha motivado una transición hacia modelos de programación basados en concurrencia, donde el software se diseña para aprovechar múltiples unidades de procesamiento [3].

Para comprender la arquitectura propuesta en este proyecto, es necesario considerar diversos conceptos fundamentales relacionados con concurrencia, paralelismo y diseño de software.

4.1. Concurrencia y paralelismo

La concurrencia se refiere a la capacidad de un sistema para gestionar múltiples tareas de forma intercalada, permitiendo que varias operaciones progresen de manera aparentemente simultánea. En contraste, el paralelismo implica la ejecución real de múltiples tareas al mismo tiempo utilizando diferentes unidades de procesamiento.

Aunque ambos conceptos están relacionados, no son equivalentes. La concurrencia se enfoca en la estructura del programa y en la forma en que las tareas se organizan y coordinan, mientras que el paralelismo se refiere a la ejecución simultánea de dichas tareas sobre múltiples procesadores o núcleos [5].

En sistemas donde existen múltiples operaciones independientes o parcialmente independientes, el uso de concurrencia y paralelismo puede reducir significativamente el tiempo total de ejecución.

4.2. Tareas CPU-bound e I/O-bound

Una distinción fundamental en el diseño de sistemas concurrentes es la diferencia entre tareas *CPU-bound* y tareas *I/O-bound*.

Las tareas *CPU-bound* son aquellas cuyo tiempo de ejecución está dominado por operaciones computacionales intensivas, como cálculos numéricos o procesamiento de datos. En estos casos, el rendimiento del sistema depende principalmente de la capacidad de procesamiento

del CPU.

Por otro lado, las tareas *I/O-bound* son aquellas cuyo tiempo de ejecución está dominado por operaciones de entrada y salida, como la lectura de sensores, el acceso a archivos o la comunicación con dispositivos externos.

En sistemas que combinan ambos tipos de tareas, suele ser necesario utilizar modelos híbridos de concurrencia que permitan optimizar tanto las operaciones de cálculo como las operaciones de entrada y salida [4].

4.3. Concurrencia en Python

El lenguaje de programación Python ofrece diferentes mecanismos para implementar concurrencia, incluyendo multithreading, programación asíncrona y multiprocessing.

Sin embargo, uno de los factores que influyen en el diseño de sistemas concurrentes en Python es la presencia del *Global Interpreter Lock* (GIL). El GIL es un mecanismo interno del intérprete que asegura que solo un hilo ejecute código Python a la vez dentro de un mismo proceso.

Aunque este mecanismo simplifica la gestión de memoria dentro del intérprete, también limita la eficiencia del uso de múltiples hilos en tareas CPU-bound [8].

Debido a esta limitación, el paralelismo en Python suele lograrse mediante el uso de múltiples procesos independientes.

4.4. AsyncIO

AsyncIO es un modelo de concurrencia basado en programación asíncrona que permite gestionar múltiples operaciones de entrada y salida sin bloquear el hilo principal de ejecución.

Este modelo se basa en un bucle de eventos (*event loop*) que coordina la ejecución de tareas asíncronas mediante el uso de corutinas. Cuando una tarea debe esperar el resultado de una operación de entrada o salida, cede temporalmente el control al bucle de eventos, permitiendo que otras tareas continúen ejecutándose.

El modelo de programación asíncrona fue formalizado en Python mediante la introducción de las palabras clave `async` y `await`, definidas en la propuesta PEP 492 [7].

Este enfoque resulta particularmente eficiente para manejar tareas I/O-bound, ya que permite solapar múltiples operaciones de entrada y salida sin necesidad de crear múltiples hilos.

4.5. Multiprocessing

Para las tareas intensivas de cómputo, Python ofrece el módulo `multiprocessing`, el cual permite ejecutar múltiples procesos independientes que se distribuyen entre los núcleos disponibles del procesador.

A diferencia del multithreading, el uso de múltiples procesos evita las limitaciones impuestas por el GIL, permitiendo que varias operaciones CPU-bound se ejecuten realmente en paralelo.

Este enfoque resulta especialmente útil en aplicaciones donde el procesamiento de datos requiere cálculos intensivos.

4.6. Ley de Amdahl

La Ley de Amdahl proporciona un modelo teórico para estimar la mejora de rendimiento que puede obtenerse al paralelizar un sistema.

Si P representa la fracción paralelizable de un programa y N representa el número de unidades de procesamiento disponibles, el speedup máximo teórico puede estimarse mediante la expresión:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Este modelo indica que la mejora total del sistema está limitada por la parte que permanece secuencial [1].

En el contexto del sistema analizado en este proyecto, una parte importante del procesamiento corresponde a operaciones independientes entre intersecciones, lo que sugiere que el sistema puede beneficiarse significativamente del uso de concurrencia y paralelismo.

5. Arquitectura propuesta

La arquitectura propuesta para el sistema de sincronización de semáforos se basa en un modelo modular orientado a objetos que permite desacoplar las distintas responsabilidades del sistema y facilitar la incorporación de mecanismos de concurrencia.

El diseño general del sistema sigue el patrón *Pipeline*, en el cual cada intersección de tráfico se procesa mediante una secuencia de etapas que transforman progresivamente la información de entrada hasta obtener el estado final del semáforo.

El pipeline de procesamiento se compone de cuatro etapas principales:

Read → Analyze → Optimize → Update

Cada etapa del pipeline corresponde a un componente independiente dentro de la arquitectura del sistema.

5.1. Estructura general del sistema

En la arquitectura propuesta, cada intersección es procesada mediante un pipeline independiente. Esto permite que múltiples intersecciones puedan ser evaluadas de manera concurrente.

Los principales componentes del sistema son los siguientes:

- **Acquisition (TrafficSensorReaderAsync)**

Este componente es responsable de la adquisición de datos provenientes de sensores de tráfico. En la implementación actual, esta etapa se simula mediante la generación de datos de conteo vehicular y marcas de tiempo.

Debido a que la lectura de sensores corresponde a una operación I/O-bound, esta etapa se implementa mediante funciones asíncronas utilizando *AsyncIO*.

- **Processing (TrafficAnalyzer)**

Este módulo se encarga del análisis de los datos de tráfico obtenidos por los sensores. A partir de los conteos vehiculares se calculan diferentes métricas, entre ellas:

- densidad vehicular
- flujo de tráfico
- longitud de la cola

Estas operaciones corresponden a tareas CPU-bound, por lo que pueden beneficiarse del uso de paralelismo mediante multiprocesamiento.

■ **Optimization (SignalOptimizer)**

El módulo de optimización es responsable de generar el plan semafórico que se aplicará a la intersección. Este plan determina parámetros como:

- tiempo de luz verde
- desfase temporal
- número de fases del semáforo

La arquitectura utiliza el patrón *Strategy* para encapsular diferentes algoritmos de optimización. En la implementación actual se incluyen dos estrategias:

- DensityBasedOptimizer
- PredictiveOptimizer

Este diseño permite incorporar nuevas estrategias de optimización sin modificar el resto del sistema.

■ **Control (TrafficControllerAsync)**

Este componente es responsable de aplicar el plan semafórico calculado al controlador de la intersección.

Al igual que la adquisición de datos, esta operación corresponde a una tarea I/O-bound, por lo que se implementa mediante funciones asíncronas utilizando *AsyncIO*.

5.2. Pipeline de intersección

El procesamiento de una intersección se encapsula dentro de la clase `IntersectionPipeline`, la cual coordina las distintas etapas del pipeline.

El flujo de procesamiento de una intersección puede describirse mediante la siguiente secuencia:

1. lectura de datos de sensores
2. análisis de métricas de tráfico
3. cálculo del plan semafórico

4. actualización del estado del semáforo

Este diseño permite aislar las responsabilidades de cada etapa y facilita la reutilización de componentes dentro del sistema.

5.3. Orquestador de ejecución

El componente encargado de coordinar el procesamiento de múltiples intersecciones es el `OrchestratorQ`.

Este módulo controla la ejecución del sistema y permite procesar un conjunto de intersecciones mediante ciclos de simulación.

El orquestador soporta dos modos de ejecución:

- **Modo asincrónico**

Cuando el número de workers es igual a uno, el sistema utiliza exclusivamente *AsyncIO*. En este modo, las operaciones I/O-bound se ejecutan de manera concurrente mientras que las tareas CPU-bound se ejecutan de forma secuencial.

- **Modo híbrido**

Cuando el número de workers es mayor que uno, el sistema combina *AsyncIO* con multiprocesamiento mediante `ProcessPoolExecutor`. En este caso, las etapas de análisis y optimización se ejecutan en procesos independientes, permitiendo paralelismo real en tareas CPU-bound.

5.4. Flujo de ejecución concurrente

En el modo híbrido, el procesamiento de un ciclo del sistema se realiza en tres fases principales:

1. lectura concurrente de datos de sensores mediante *AsyncIO*
2. procesamiento de análisis y optimización utilizando un pool de procesos
3. actualización concurrente de los controladores de semáforos

Este modelo permite aprovechar tanto la concurrencia en operaciones de entrada/salida como el paralelismo en operaciones computacionales, mejorando significativamente el rendimiento del sistema.

5.5. Ventajas de la arquitectura

La arquitectura propuesta ofrece diversas ventajas desde el punto de vista de diseño de software y rendimiento computacional:

- separación clara de responsabilidades entre los distintos componentes del sistema
- posibilidad de ejecutar múltiples pipelines de forma concurrente
- extensibilidad mediante la incorporación de nuevas estrategias de optimización
- mejor aprovechamiento de los recursos de hardware disponibles
- reducción del tiempo total de procesamiento del sistema

Gracias a estas características, la arquitectura permite mejorar tanto la escalabilidad como la mantenibilidad del sistema de control de tráfico.

6. Estrategia de concurrencia propuesta

El sistema de procesamiento de intersecciones analizado en este proyecto presenta un pipeline compuesto por múltiples etapas con características computacionales distintas. Algunas etapas están dominadas por operaciones de entrada y salida, mientras que otras requieren procesamiento intensivo de datos.

Debido a esta combinación de tareas, la estrategia de concurrencia propuesta se basa en un modelo híbrido que combina programación asíncrona y multiprocesamiento. Este enfoque permite aprovechar tanto la concurrencia en operaciones I/O-bound como el paralelismo en tareas CPU-bound [4].

6.1. Clasificación de las etapas del pipeline

El pipeline de procesamiento del sistema puede representarse mediante las siguientes etapas:

Read $\rightarrow$ Analyze $\rightarrow$ Optimize $\rightarrow$ Update

Cada una de estas etapas presenta características computacionales diferentes. En la Tabla 1 se muestra la clasificación de las etapas del pipeline según el tipo de tarea que realizan.

Etapas	Tipo de tarea	Descripción
Read	I/O-bound	Lectura de datos de sensores de tráfico
Analyze	CPU-bound	Cálculo de métricas vehiculares
Optimize	CPU-bound	Generación del plan semafórico
Update	I/O-bound	Actualización del estado del semáforo

Cuadro 1: Clasificación de las etapas del pipeline

Las etapas *Read* y *Update* corresponden a operaciones de entrada y salida, mientras que las etapas *Analyze* y *Optimize* implican cálculos computacionales más intensivos. Esta separación permite identificar qué partes del sistema pueden beneficiarse de concurrencia y cuáles requieren paralelismo.

6.2. Uso de AsyncIO para tareas I/O-bound

Para las operaciones de entrada y salida, el sistema utiliza el modelo de programación asíncrona proporcionado por *AsyncIO*. Este modelo permite gestionar múltiples operaciones de entrada y salida sin bloquear el hilo principal de ejecución mediante el uso de corutinas y un bucle de eventos.

Cuando una tarea requiere esperar una respuesta externa, el control se devuelve temporalmente al bucle de eventos, permitiendo que otras tareas continúen ejecutándose. Este enfoque resulta particularmente eficiente en sistemas donde una parte significativa del tiempo de ejecución corresponde a latencias de entrada y salida [7].

En la arquitectura propuesta, AsyncIO se utiliza principalmente para las etapas del pipeline relacionadas con la lectura de sensores y la actualización de los controladores de semáforos.

6.3. Uso de multiprocesamiento para tareas CPU-bound

Las etapas de análisis y optimización del pipeline requieren procesamiento computacional intensivo. En Python, este tipo de tareas puede verse limitado por la presencia del *Global Interpreter Lock* (GIL), el cual impide que múltiples hilos ejecuten código Python de manera simultánea dentro de un mismo proceso.

Para superar esta limitación, el sistema utiliza multiprocesamiento mediante el uso de `ProcessPoolExecutor`. Este mecanismo permite ejecutar funciones en procesos independientes, distribuyendo la carga de trabajo entre múltiples núcleos del procesador.

Al utilizar procesos separados, el sistema evita las restricciones del GIL y permite obtener paralelismo real en tareas CPU-bound [8].

6.4. Modelo híbrido de ejecución

La arquitectura del sistema combina ambos enfoques mediante un modelo híbrido de concurrencia.

Durante la ejecución de un ciclo del sistema, el procesamiento se organiza en tres fases principales:

1. lectura concurrente de datos de sensores mediante AsyncIO
2. ejecución paralela de las etapas de análisis y optimización mediante un pool de procesos
3. actualización concurrente de los controladores de semáforos

Este modelo permite aprovechar simultáneamente las ventajas de la programación asíncrona y del paralelismo basado en procesos [5].

6.5. Justificación técnica

La elección de una arquitectura híbrida responde a las características del sistema analizado. En primer lugar, las operaciones I/O-bound representan una parte importante del tiempo

de ejecución del sistema, por lo que el uso de programación asíncrona permite solapar estas latencias.

En segundo lugar, las operaciones CPU-bound pueden beneficiarse del uso de múltiples núcleos del procesador mediante multiprocesamiento. Este enfoque permite distribuir el trabajo entre diferentes unidades de procesamiento y mejorar el rendimiento global del sistema [2].

De acuerdo con la Ley de Amdahl, el speedup máximo que puede obtenerse al paralelizar un sistema depende de la fracción del programa que puede ejecutarse en paralelo [1]. En el caso del sistema analizado, cada intersección puede procesarse de manera independiente, lo que permite aprovechar de forma efectiva los mecanismos de concurrencia y paralelismo propuestos.

En conjunto, esta estrategia permite reducir el tiempo total de ejecución del sistema y mejorar su capacidad de escalamiento conforme aumenta el número de intersecciones procesadas.

7. Metodología de evaluación

7.1. Objetivo del benchmark

El objetivo del benchmark es evaluar cuantitativamente la ganancia de rendimiento obtenida al pasar de una arquitectura secuencial (Estado P) a una arquitectura concurrente (Estado Q) en el sistema de procesamiento de intersecciones semafóricas.

La evaluación considera distintas configuraciones de concurrencia y analiza cómo varía el rendimiento conforme aumenta el número de intersecciones procesadas en cada ciclo.

7.2. Configuración experimental

La Tabla 2 resume los parámetros utilizados durante los experimentos.

Parámetro	Valor
Ciclos por prueba	30
Carga CPU (<code>cpu_work</code>)	30 000 iteraciones
Latencia I/O lectura	25 ms
Latencia I/O update	10 ms
Semilla	1234
CPU cores disponibles	20
Optimizer (Strategy)	density

Cuadro 2: Configuración experimental utilizada en los benchmarks

7.3. Modos de ejecución evaluados

Se evaluaron tres configuraciones del sistema:

- **Estado P — Secuencial**

Pipeline completo ejecutado de manera estrictamente secuencial para cada intersección. No se utiliza concurrencia.

- **Estado Q async-only**

Implementación basada en AsyncIO que permite solapar operaciones de entrada/salida. Las tareas CPU-bound continúan ejecutándose en el hilo principal.

- **Estado Q full**

Implementación concurrente completa que combina AsyncIO con `ProcessPoolExecutor` utilizando cuatro workers para ejecutar las tareas CPU-bound en paralelo.

7.4. Resultados experimentales

Los experimentos se ejecutaron variando el número de intersecciones procesadas en cada ciclo:

$$n = \{10, 25, 50, 100, 200\}$$

7.4.1. Tiempos promedio de ejecución

La Tabla 3 muestra los tiempos promedio de ejecución por ciclo para cada configuración.

n	T_P (s)	$T_{Q_{async}}$ (s)	$T_{Q_{full}}$ (s)
10	0.377	0.054	0.044
25	0.938	0.080	0.051
50	1.893	0.121	0.062
100	3.763	0.201	0.083
200	7.562	0.370	0.125

Cuadro 3: Tiempos promedio de ejecución por ciclo

Los resultados muestran que el tiempo de ejecución del sistema secuencial crece aproximadamente de forma lineal con el número de intersecciones procesadas, mientras que las versiones concurrentes presentan un crecimiento significativamente menor.

7.4.2. Speedup

Además del speedup respecto al sistema secuencial, se incluye una tercera métrica denominada *speedup paralelo*. Esta métrica mide la mejora obtenida al pasar de la versión concurrente basada únicamente en AsyncIO (Q_{async}) a la versión concurrente completa que incorpora paralelismo mediante múltiples procesos (Q_{full}).

Formalmente, el speedup paralelo se define como:

$$S_{paralelo} = \frac{T_{Q_{async}}}{T_{Q_{full}}}$$

Esta métrica permite aislar el impacto específico del paralelismo CPU introducido por el uso de `ProcessPoolExecutor`.

El speedup se calcula como:

$$Speedup = \frac{T_P}{T_Q}$$

La Tabla 4 muestra los speedups obtenidos para las dos variantes concurrentes.

n	Speedup async	Speedup full	Speedup paralelo
10	7.0x	8.6x	1.2x
25	11.7x	18.4x	1.6x
50	15.6x	30.5x	2.0x
100	18.7x	45.3x	2.4x
200	20.4x	60.5x	3.0x

Cuadro 4: Speedup del sistema concurrente

Los resultados muestran que el speedup aumenta conforme crece el tamaño del problema. Esto indica que la arquitectura concurrente aprovecha mejor los recursos computacionales cuando la carga de trabajo es mayor.

7.4.3. Eficiencia del paralelismo

La eficiencia mide qué tan bien se aprovechan los workers del `ProcessPoolExecutor`:

$$E = \frac{S_{paralelo}}{p}$$

donde $p = 4$ corresponde al número de workers utilizados.

n	Eficiencia
10	0.31
25	0.39
50	0.49
100	0.61
200	0.74

Cuadro 5: Eficiencia del paralelismo

La eficiencia aumenta conforme crece el número de intersecciones, debido a que el overhead de coordinación del pool de procesos se amortiza cuando la carga de trabajo es mayor.

7.5. Análisis de resultados

7.5.1. Análisis de profiling del sistema

Con el objetivo de identificar los principales cuellos de botella del sistema base (Estado P), se realizó un análisis de *profiling* utilizando la herramienta `cProfile` sobre la implementación

secuencial del pipeline.

El profiling se ejecutó sobre el script `CodigoSecuencial/run_p0.py`, el cual implementa el ciclo de procesamiento secuencial de intersecciones. El experimento se ejecutó con los parámetros mostrados en la Tabla 6.

Parámetro	Valor
Número de intersecciones	$n = 50$
Ciclos de simulación	1
Carga de CPU	30 000 iteraciones
Latencia de lectura (I/O)	25 ms
Latencia de actualización (I/O)	10 ms
Semilla aleatoria	1234

Cuadro 6: Configuración del experimento de profiling.

El profiling se ejecutó mediante el siguiente comando:

```
cd profiling
python3 profile_sequential.py --n 50 --cycles 1
```

Este script ejecuta el ciclo secuencial completo y registra las funciones ordenadas por tiempo acumulado (*cumtime*) y tiempo propio (*tottime*).

Pipeline del estado P El sistema secuencial procesa cada intersección mediante un pipeline compuesto por cuatro etapas:

$$read_sensor(i) \rightarrow analyze(data) \rightarrow optimize(features) \rightarrow update_light(i, plan)$$

Cada una de estas etapas presenta diferentes características computacionales, como se muestra en la Tabla 7.

Etapas	Tipo	Simulación
<code>read_sensor</code>	I/O-bound	<code>time.sleep(25ms)</code> + generación de datos
<code>analyze</code>	CPU-bound	bucle de 30 000 iteraciones con funciones trigonométricas
<code>optimize</code>	CPU-light	operaciones aritméticas simples
<code>update_light</code>	I/O-bound	<code>time.sleep(10ms)</code> + actualización de estado

Cuadro 7: Clasificación computacional de las etapas del pipeline.

En el estado secuencial, las n intersecciones se procesan una tras otra, por lo que el tiempo total de ejecución puede aproximarse como:

$$T_P = \sum_{i=1}^n (t_{read} + t_{analyze} + t_{optimize} + t_{update})$$

Considerando los valores simulados del sistema, esto puede aproximarse como:

$$T_P \approx n \times 41,5 \text{ ms}$$

Resultados del profiling El profiling del sistema secuencial produjo los siguientes resultados principales:

- Tiempo total del ciclo: 2.084 s
- Número total de llamadas a funciones: 3 002 723

Las funciones con mayor tiempo propio (*tottime*) fueron:

Función	Llamadas	Tiempo propio (s)
time.sleep	100	1.758
analyze	50	0.184
math.cos	1 500 000	0.071
math.sin	1 500 000	0.068

Cuadro 8: Funciones dominantes por tiempo propio (*tottime*).

Asimismo, al analizar el tiempo acumulado (*cumtime*) se observa que las operaciones de lectura y actualización dominan el tiempo total del sistema, ya que incluyen las latencias simuladas de entrada y salida.

Interpretación de los resultados El análisis del profiling muestra que la mayor parte del tiempo de ejecución del sistema secuencial corresponde a operaciones de entrada/salida simuladas mediante `time.sleep`. En particular, las llamadas a esta función representan aproximadamente 1.758 segundos del total de 2.084 segundos de ejecución.

Esto indica que el sistema base está dominado por operaciones *I/O-bound*, mientras que las operaciones *CPU-bound* (asociadas al análisis de datos) representan una fracción menor del tiempo total.

Este resultado es consistente con el modelo de procesamiento del pipeline, donde cada intersección requiere esperar latencias externas antes de continuar con el siguiente paso.

Implicaciones para el diseño concurrente Los resultados del profiling justifican el uso de una arquitectura concurrente en el Estado Q. Dado que las operaciones dominantes del sistema corresponden a tareas I/O-bound, el uso de programación asíncrona mediante AsyncIO permite solapar múltiples operaciones de lectura y actualización de sensores.

Adicionalmente, las etapas CPU-bound del pipeline pueden paralelizarse mediante multiprocesamiento, permitiendo aprovechar múltiples núcleos de CPU.

En conjunto, este análisis proporciona evidencia empírica sobre el cuello de botella del sistema secuencial y fundamenta la estrategia de concurrencia utilizada en la arquitectura optimizada.

7.5.2. Impacto de AsyncIO

El primer cambio arquitectónico evaluado consiste en introducir concurrencia mediante AsyncIO, permitiendo solapar operaciones de entrada y salida entre múltiples intersecciones.

La Figura 1 muestra la comparación global entre el sistema secuencial (Estado P) y la implementación concurrente.

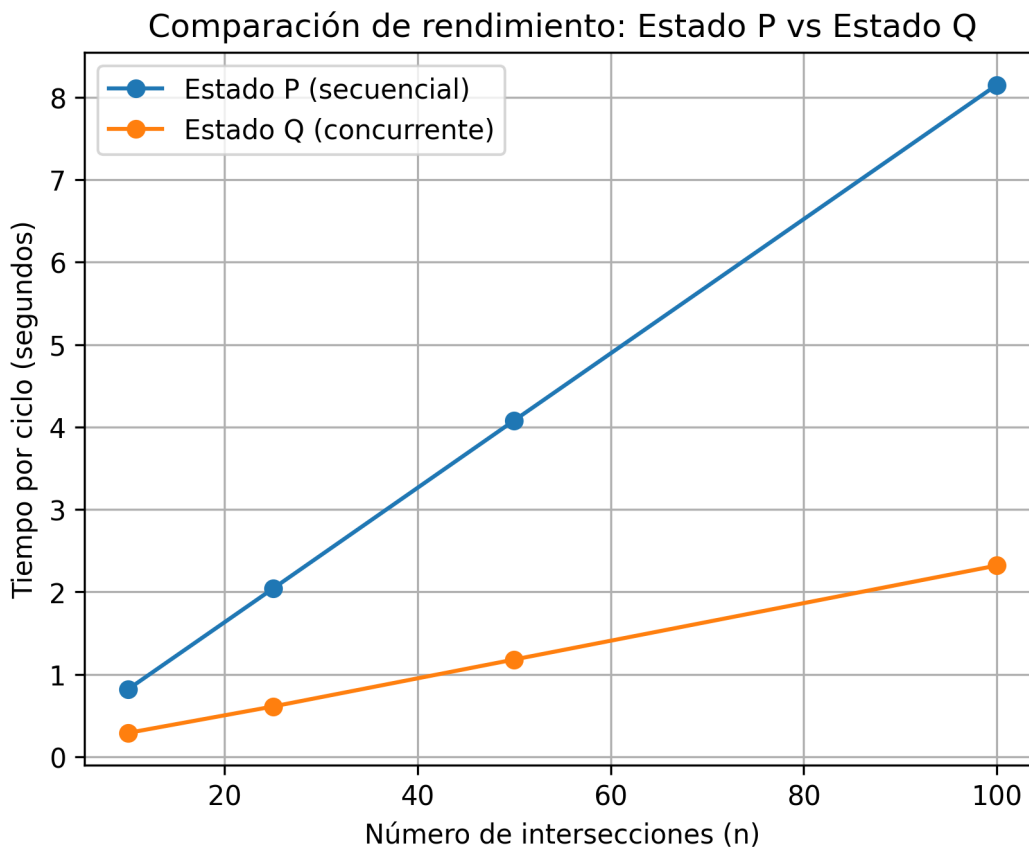


Figura 1: Comparación general del tiempo de ejecución entre el sistema secuencial (Estado P) y el sistema concurrente (Estado Q).

El profiling del sistema secuencial indica que aproximadamente el 91.6 % del tiempo de ejecución corresponde a operaciones I/O simuladas (lectura de sensores y actualización de semáforos), mientras que solo el 8.3 % corresponde a trabajo CPU.

AsyncIO permite solapar estas operaciones I/O, de modo que mientras una intersección espera una respuesta de sensor, otra intersección puede continuar su procesamiento.

La Figura 2 muestra la comparación detallada de los tiempos de ejecución para distintos tamaños del problema.

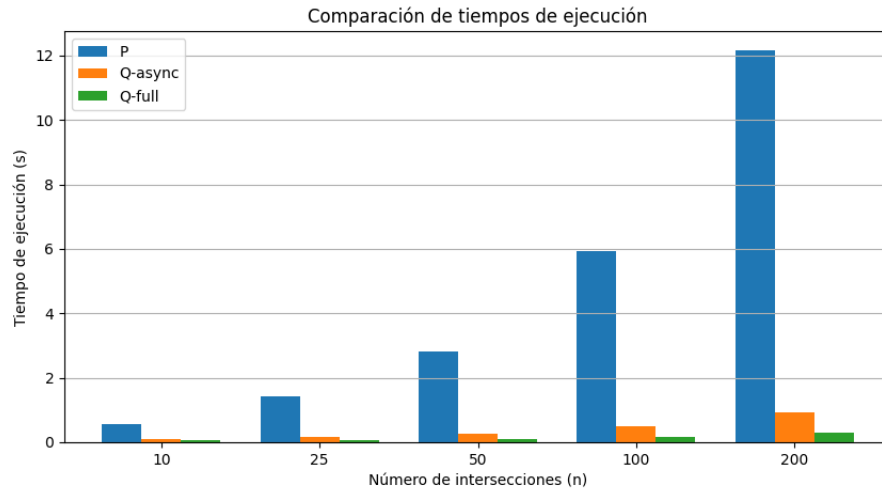


Figura 2: Comparación de tiempos de ejecución entre las implementaciones secuencial y concurrente.

Se observa que el tiempo de ejecución del sistema secuencial crece aproximadamente de forma lineal con el número de intersecciones, mientras que la versión basada en AsyncIO reduce drásticamente el tiempo total al solapar las operaciones de entrada y salida.

7.5.3. Impacto de Multiprocessing

El segundo cambio arquitectónico consiste en introducir paralelismo real para las tareas CPU-bound mediante `ProcessPoolExecutor`.

En esta configuración, el trabajo de optimización del plan semafórico se distribuye entre múltiples procesos independientes.

La Figura 3 muestra el speedup obtenido al comparar el sistema secuencial con la arquitectura concurrente.

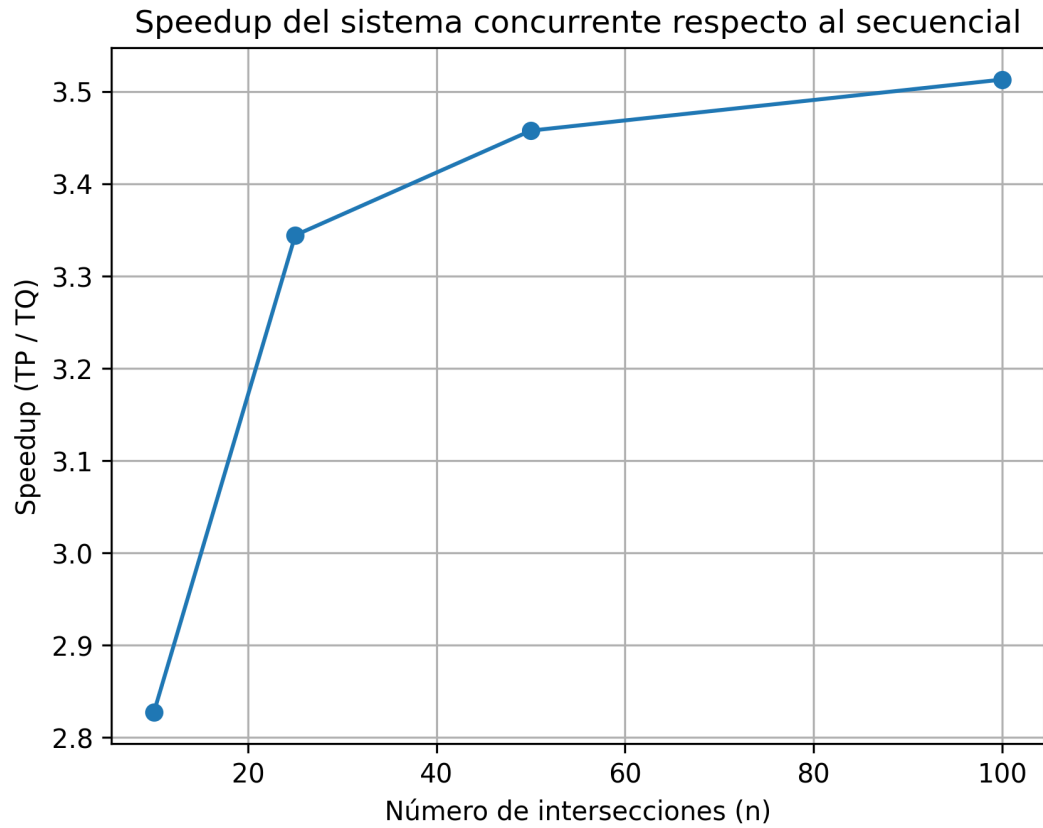


Figura 3: Speedup del sistema concurrente respecto al sistema secuencial.

La Figura 4 muestra una visualización adicional del speedup generado a partir de los scripts de análisis del repositorio.

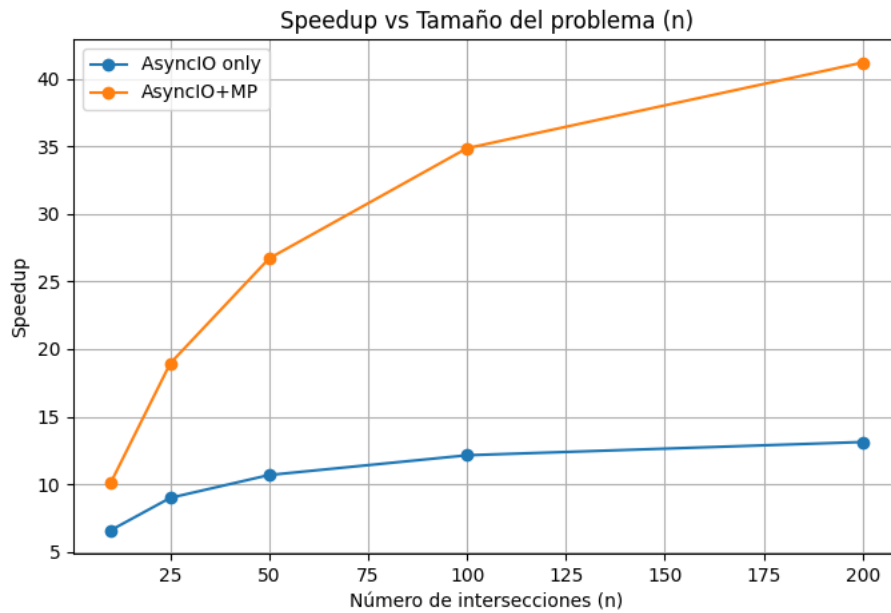


Figura 4: Evolución del speedup conforme aumenta el número de intersecciones procesadas.

Los resultados muestran que el speedup combinado de AsyncIO y multiprocessing puede alcanzar valores superiores a 60x cuando el número de intersecciones es suficientemente grande.

Este comportamiento se explica porque la arquitectura concurrente permite aprovechar dos niveles distintos de paralelismo:

- Concurrencia I/O mediante AsyncIO.
- Paralelismo CPU mediante múltiples procesos.

7.5.4. Escalabilidad

El comportamiento del sistema concurrente frente al crecimiento del tamaño del problema puede analizarse mediante su escalabilidad.

La Figura 5 muestra el throughput del sistema conforme aumenta el número de intersecciones.

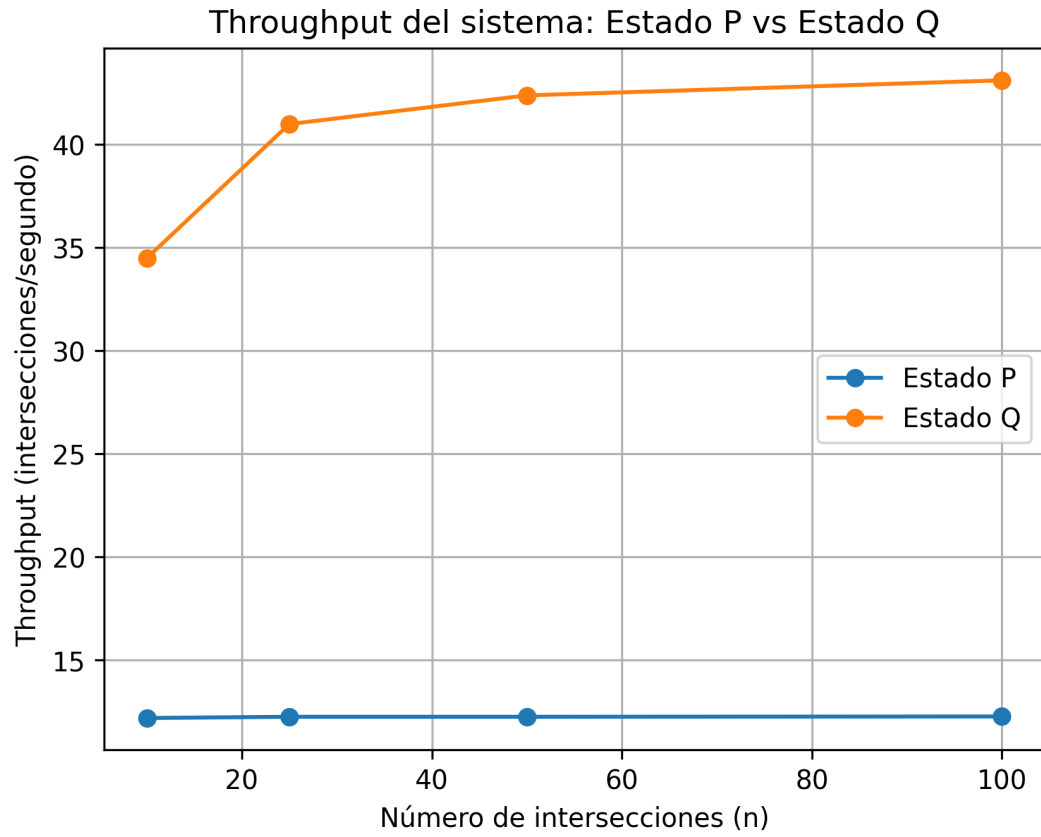


Figura 5: Throughput del sistema concurrente.

La Figura 6 muestra una visualización adicional del throughput generada por los scripts de visualización.

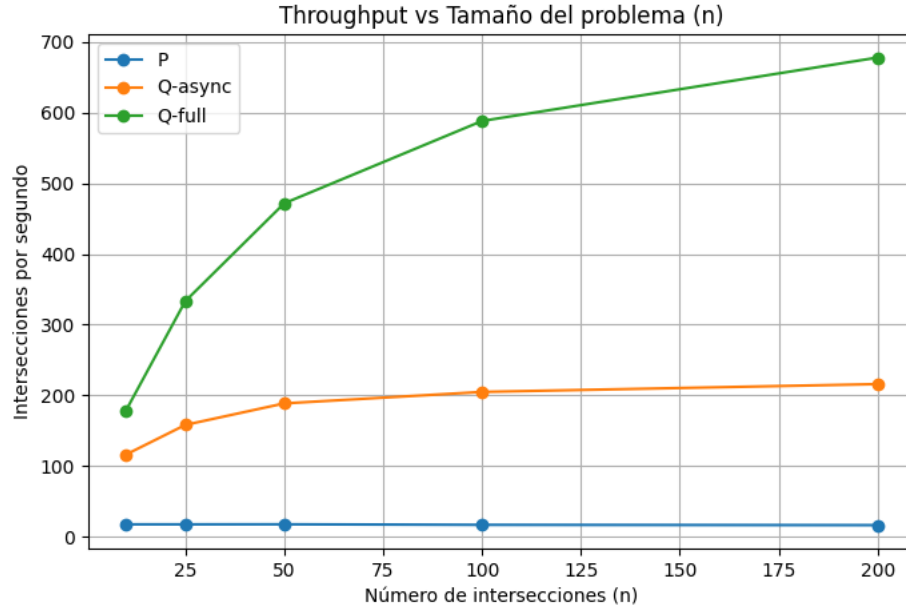


Figura 6: Throughput en función del número de intersecciones procesadas.

Los resultados muestran que el throughput aumenta significativamente conforme crece el número de intersecciones, lo cual indica que la arquitectura concurrente es capaz de procesar más intersecciones por unidad de tiempo.

En particular, multiplicar por veinte el número de intersecciones procesadas no implica un aumento proporcional del tiempo de ejecución, lo cual confirma que el sistema escala eficientemente.

7.6. Ley de Amdahl

La Ley de Amdahl establece el límite teórico del speedup alcanzable mediante paralelización [1]:

$$S = \frac{1}{(1 - \alpha) + \frac{\alpha}{p}}$$

donde α representa la fracción paralelizable del programa y p el número de unidades de ejecución.

A partir de los resultados experimentales se estima que aproximadamente el 95 % del tiempo de ejecución del sistema puede paralelizarse.

La fracción serial restante corresponde principalmente a operaciones de orquestación del pipeline, incluyendo:

- planificación de coroutines en AsyncIO,
- serialización de datos para el pool de procesos,
- sincronización de resultados.

Estos factores introducen un overhead inevitable que limita el speedup máximo alcanzable.

7.7. Síntesis de hallazgos

Los resultados experimentales permiten identificar varios hallazgos importantes:

- AsyncIO produce la mayor parte de la mejora de rendimiento, ya que el sistema original es predominantemente I/O-bound.
- El uso de multiprocesamiento permite reducir el tiempo dedicado a las tareas CPU-bound, distribuyendo el trabajo entre múltiples procesos.
- La combinación de ambos mecanismos permite alcanzar speedups de hasta 60x respecto al sistema secuencial.
- El sistema presenta un comportamiento de escalabilidad favorable, ya que el tiempo de ejecución crece mucho más lentamente que el número de intersecciones procesadas.

En conjunto, estos resultados demuestran que la arquitectura concurrente propuesta permite mejorar significativamente el rendimiento del sistema y escalar hacia redes de tráfico de mayor tamaño.

7.8. Reproducibilidad

Los experimentos pueden reproducirse ejecutando el siguiente script incluido en el repositorio:

```
cd benchmarks
python3 compare_P_vs_Q.py
```

Este script ejecuta los tres modos de ejecución evaluados, calcula promedios sobre múltiples ciclos y genera el archivo `results.csv` con los datos experimentales.

8. Plan de trabajo

El desarrollo del proyecto se organiza en una serie de fases que permiten avanzar de forma estructurada desde el análisis del sistema original hasta la evaluación del rendimiento de la arquitectura propuesta.

Cada fase del plan de trabajo se enfoca en una etapa específica del proceso de diseño, implementación y validación del sistema.

8.1. Fase 1: Análisis del sistema base

En la primera fase se analiza la implementación inicial del sistema (Estado P), con el objetivo de comprender su arquitectura y el flujo de procesamiento de las intersecciones.

Durante esta etapa se identifican las etapas principales del pipeline de procesamiento y se analizan los posibles cuellos de botella del modelo secuencial.

Las actividades principales de esta fase incluyen:

- análisis de la estructura del sistema secuencial
- identificación de las etapas del pipeline de procesamiento
- evaluación preliminar del rendimiento del sistema base

8.2. Fase 2: Diseño de la arquitectura modular

En esta fase se diseña una nueva arquitectura del sistema basada en programación orientada a objetos y patrones de diseño.

El objetivo de esta etapa es desacoplar las responsabilidades del sistema y estructurar el procesamiento de las intersecciones mediante un pipeline modular.

Entre las actividades realizadas se encuentran:

- definición de los componentes principales del sistema
- diseño del pipeline de procesamiento de intersecciones
- incorporación del patrón de diseño *Strategy* para la optimización del plan semafórico

8.3. Fase 3: Implementación de la arquitectura concurrente

Una vez definida la arquitectura modular, se implementa una versión concurrente del sistema (Estado Q).

Durante esta fase se integran mecanismos de concurrencia que permiten procesar múltiples intersecciones de manera simultánea.

Las principales actividades de esta fase incluyen:

- implementación de operaciones asíncronas mediante AsyncIO
- integración de multiprocesamiento para tareas CPU-bound
- desarrollo del orquestador encargado de coordinar la ejecución del sistema

8.4. Fase 4: Desarrollo del sistema de benchmarking

En esta etapa se desarrolla un conjunto de herramientas que permiten evaluar el rendimiento del sistema bajo diferentes configuraciones.

Estas herramientas automatizan la ejecución de experimentos y recopilan las métricas necesarias para comparar el comportamiento del sistema secuencial y el sistema concurrente.

Las actividades principales incluyen:

- implementación de scripts de benchmarking
- recopilación automática de métricas de rendimiento
- generación de archivos de resultados para su análisis

8.5. Fase 5: Evaluación experimental

En la fase final del proyecto se ejecutan los experimentos definidos en la metodología de evaluación.

Los resultados obtenidos permiten analizar el impacto de la concurrencia en el rendimiento del sistema.

Entre las actividades realizadas se encuentran:

- ejecución de experimentos con diferentes tamaños de red
- cálculo de métricas como tiempo de ejecución, throughput y speedup
- análisis de los resultados obtenidos

Esta fase permite validar la hipótesis de que una arquitectura basada en concurrencia puede mejorar significativamente el rendimiento del sistema.

9. Estructura del repositorio

El código fuente del proyecto se encuentra disponible en el repositorio:

`https://github.com/RamsesCamas/MCIT\_LOG\_PROG\_Proyecto\_Final.git`

El repositorio está organizado en diferentes módulos que separan la implementación del sistema, las herramientas de evaluación y la documentación del proyecto. Esta organización facilita la comprensión del código y permite aislar las distintas responsabilidades del sistema.

A continuación se presenta la estructura general del repositorio:

`MCIT_LOG_PROG_Proyecto_Final/`

`CodigoSecuencial/`

`run_p0.py`

`CodigoConcurrente/`

`src/`

`main.py`

`orchestrator_q.py`

`pipeline.py`

`acquisition.py`

`processing.py`

`control.py`

`workers.py`

`config.py`

`domain.py`

`optimization/`

`interfaces.py`

`strategies.py`

`factory.py`

`benchmarks/`

`compare_P_vs_Q.py`

`results.csv`

`profiling/`

`profile_sequential.py`

ANALYSIS.md

visualizacion/
generar_graficas.py

Documentacion/
reporte.tex
reporte.pdf
graficas/

9.1. Implementación secuencial

El directorio `CodigoSecuencial` contiene la implementación original del sistema, denominada *Estado P*. En esta versión las intersecciones de tráfico se procesan de manera estrictamente secuencial, lo que implica que cada etapa del pipeline se ejecuta una después de otra para cada intersección.

El archivo principal de esta implementación es:

- `run_p0.py`: script encargado de ejecutar el pipeline secuencial completo de procesamiento de intersecciones.

Esta versión sirve como punto de referencia para comparar el rendimiento del sistema concurrente desarrollado posteriormente.

9.2. Implementación concurrente

La implementación concurrente del sistema se encuentra en el directorio `CodigoConcurrente/src`. Esta versión corresponde al *Estado Q* del sistema y está diseñada para procesar múltiples intersecciones de manera concurrente.

El módulo está organizado en distintos archivos que separan las responsabilidades del sistema:

- `main.py`: punto de entrada del sistema concurrente.
- `orchestrator_q.py`: componente encargado de coordinar la ejecución del pipeline concurrente.
- `pipeline.py`: implementación del pipeline de procesamiento de intersecciones.

- `acquisition.py`: módulo responsable de la adquisición de datos de tráfico.
- `processing.py`: módulo encargado del análisis de los datos recolectados.
- `control.py`: módulo encargado de aplicar las decisiones de control del sistema.
- `workers.py`: implementación de los trabajadores encargados de ejecutar tareas concurrentes.
- `config.py`: archivo de configuración del sistema.
- `domain.py`: definiciones de las entidades del dominio del problema.

Adicionalmente, el subdirectorio `optimization` contiene las estrategias utilizadas para calcular los planes semafóricos, las interfaces asociadas y una fábrica de estrategias que permite seleccionar dinámicamente el algoritmo de optimización utilizado.

9.3. Herramientas de evaluación

El directorio `benchmarks` contiene los scripts utilizados para comparar el rendimiento del sistema secuencial y del sistema concurrente.

En particular:

- `compare_P_vs_Q.py`: script que ejecuta experimentos de benchmarking para comparar ambas implementaciones.
- `results.csv`: archivo que almacena los resultados experimentales obtenidos durante las pruebas.

Estos datos se utilizan posteriormente para generar las métricas de rendimiento presentadas en la sección de evaluación.

9.4. Herramientas de análisis

El directorio `profiling` contiene scripts utilizados para analizar el comportamiento del sistema secuencial mediante herramientas de perfilado de rendimiento.

Estos análisis permiten identificar los principales cuellos de botella del sistema original y sirven como base para justificar la introducción de concurrencia en la arquitectura propuesta.

9.5. Visualización de resultados

El directorio `visualizacion` contiene el script `generar_graficas.py`, el cual se utiliza para generar las gráficas de rendimiento presentadas en este documento, incluyendo las comparaciones de tiempo de ejecución, speedup y throughput.

9.6. Documentación del proyecto

Finalmente, el directorio `Documentacion` contiene el reporte técnico del proyecto, así como los archivos asociados al documento en formato $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ y las gráficas utilizadas para ilustrar los resultados experimentales.

10. Conclusiones

El objetivo de este trabajo fue evaluar el impacto de introducir concurrencia en un sistema de procesamiento de intersecciones semafóricas, comparando una arquitectura secuencial (Estado P) con una arquitectura concurrente basada en AsyncIO y multiprocesamiento (Estado Q).

La pregunta de investigación planteada en la Sección 2.4 buscaba determinar si la introducción de concurrencia permitiría mejorar el rendimiento y la escalabilidad del sistema conforme aumenta el número de intersecciones procesadas.

Los resultados experimentales muestran que la arquitectura concurrente logra mejoras significativas de rendimiento respecto a la versión secuencial. En particular, la introducción de AsyncIO permite solapar operaciones de entrada y salida, reduciendo drásticamente el tiempo de espera asociado a las lecturas de sensores y actualizaciones de señales. Este cambio produce speedups de entre 7x y 20x dependiendo del tamaño del problema.

Adicionalmente, la incorporación de multiprocesamiento mediante `ProcessPoolExecutor` permite paralelizar las tareas CPU-bound del pipeline, generando un factor adicional de aceleración. La combinación de concurrencia para I/O y paralelismo para CPU permite alcanzar speedups de hasta 60x en los escenarios evaluados.

El análisis de profiling mostró que aproximadamente el 91.6 % del tiempo de ejecución del sistema secuencial corresponde a operaciones I/O simuladas, lo que explica por qué el uso de AsyncIO tiene un impacto tan significativo en el rendimiento global. Asimismo, el comportamiento de los resultados es consistente con la Ley de Amdahl, ya que el speedup máximo observado está limitado por el pequeño porcentaje de operaciones que permanecen secuenciales.

En términos de escalabilidad, los resultados indican que el tiempo de ejecución del sistema concurrente crece mucho más lentamente que el número de intersecciones procesadas. Esto sugiere que la arquitectura propuesta es capaz de manejar redes de tráfico significativamente más grandes sin un aumento proporcional en el tiempo de procesamiento.

No obstante, este estudio presenta algunas limitaciones. En particular, los experimentos se realizaron utilizando datos simulados mediante funciones de espera (`time.sleep`) para representar latencias de sensores y procesamiento. Aunque este enfoque permite controlar el comportamiento del sistema y reproducir los experimentos de manera consistente, no refleja completamente la complejidad de un entorno de tráfico real con sensores físicos, variabilidad en la comunicación y eventos impredecibles.

Como trabajo futuro, sería deseable extender este sistema para operar con fuentes de datos reales provenientes de sensores de tráfico o simuladores de movilidad urbana. Asimismo,

podría explorarse el uso de otros modelos de concurrencia, como arquitecturas basadas en actores o sistemas distribuidos que permitan procesar redes de tráfico a escala metropolitana.

En conjunto, los resultados obtenidos demuestran que la arquitectura concurrente propuesta no solo mejora significativamente el rendimiento del sistema, sino que también proporciona una base sólida para el desarrollo de sistemas de control de tráfico más escalables y eficientes.

11. Referencias

Referencias

- [1] Amdahl, G. (1967). *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. AFIPS Conference Proceedings. Disponible en: <https://dl.acm.org/doi/pdf/10.1145/1465482.1465560>
- [2] Hill, M. D., & Marty, M. R. (2008). *Amdahl's Law in the Multicore Era*. IEEE Computer. Disponible en: https://research.cs.wisc.edu/multifacet/papers/ieeecomputer08_amdahl.pdf
- [3] Sutter, H. (2005). *The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software*. Dr. Dobbs's Journal. Disponible en: https://www.math.pku.edu.cn/teachers/qiuzy/plan/lits/free_lunch_over.pdf
- [4] McCool, M., Reinders, J., & Robison, A. (2012). *Structured Parallel Programming: Patterns for Efficient Computation*. Morgan Kaufmann. Disponible en: <https://www.cs.cmu.edu/~15210/docs/book.pdf>
- [5] Herlihy, M., & Shavit, N. (2008). *The Art of Multiprocessor Programming*. Morgan Kaufmann. Disponible en: <https://cs.brown.edu/~mph/HerlihyShavitBook.pdf>
- [6] Dean, J., & Barroso, L. (2013). *The Tail at Scale*. Communications of the ACM. Disponible en: <https://cacm.acm.org/magazines/2013/2/160173-the-tail-at-scale/pdf>
- [7] Selivanov, Y. (2015). *PEP 492 – Coroutines with async and await syntax*. Python Enhancement Proposal. Disponible en: <https://peps.python.org/pep-0492/>
- [8] Beazley, D. (2010). *Understanding the Python GIL*. PyCon Conference Paper. Disponible en: <https://www.dabeaz.com/GIL/gil.pdf>